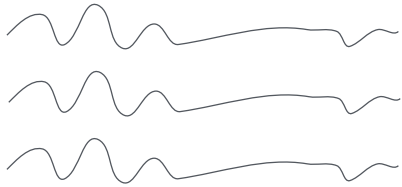
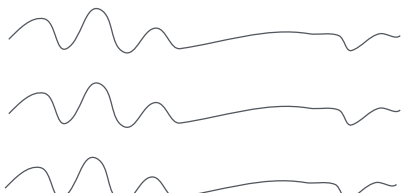


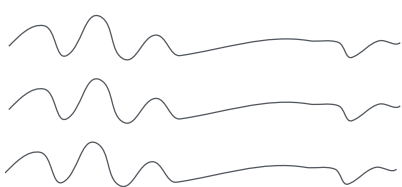
About



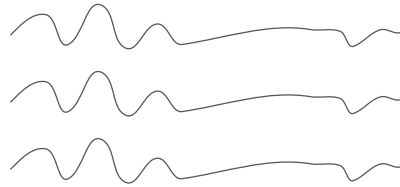
Project Config



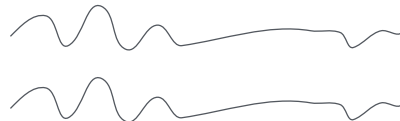
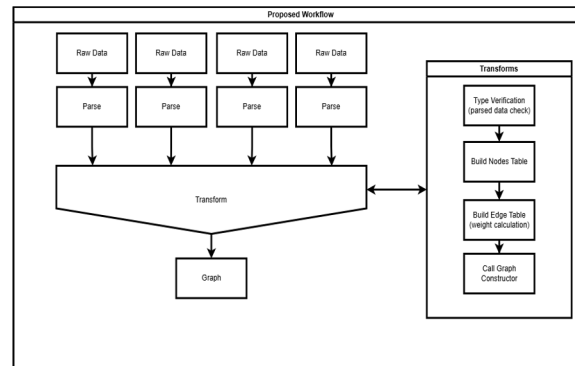
Kaiaulu Functionality



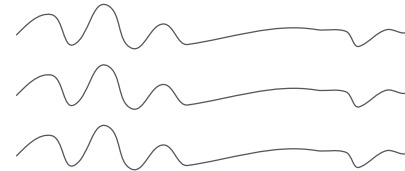
Standardization



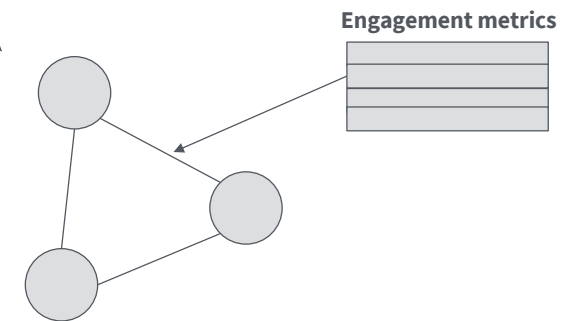
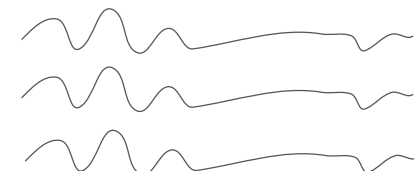
Diagram



Related Vignettes



Weight Integration Example

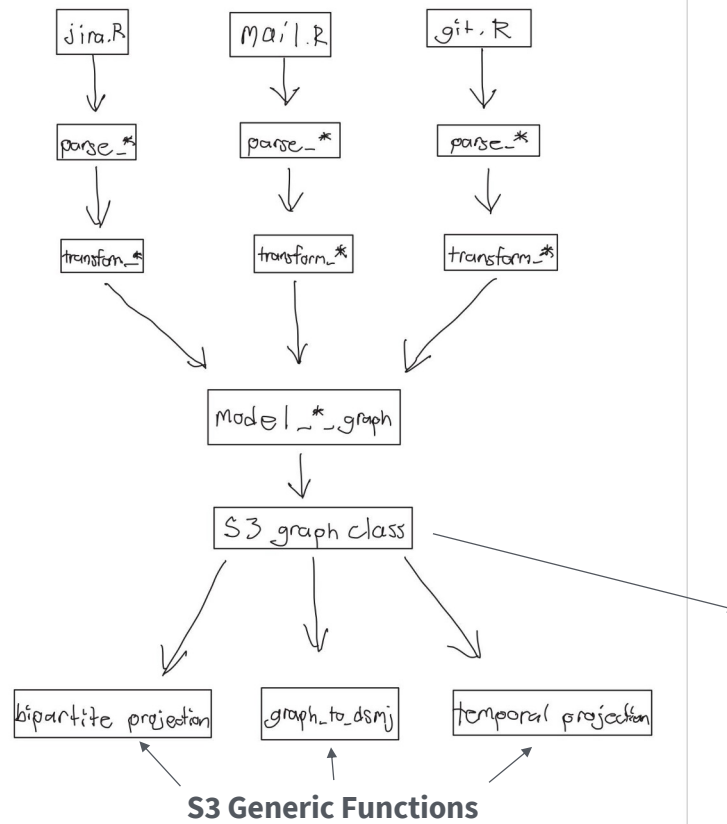


About

Project Config

Kaiaulu Functionality

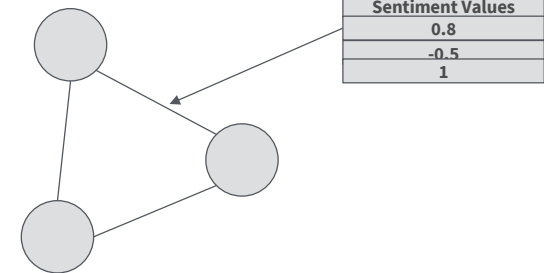
Diagram



Related Vignettes

Weight Integration Example

Engagement metrics
data.table output



Network Transform: : CHEAT SHEET



About

Kaiaulu's analysis pipelines convert raw data sources including git logs, mailing lists, and issue trackers into graphs for downstream analysis. This work introduces a standardized S3 class system for Kaiulu's graph model, replacing inconsistent graph construction with two well-defined constructors that encode graph properties such as directionality, weight, and bipartiteness directly in the object's class. Consumer functions dispatch on these classes, meaning new analyses can be implemented as S3 methods that automatically work with any graph produced by the existing transform functions without requiring changes to those transforms.

Project Config Setup

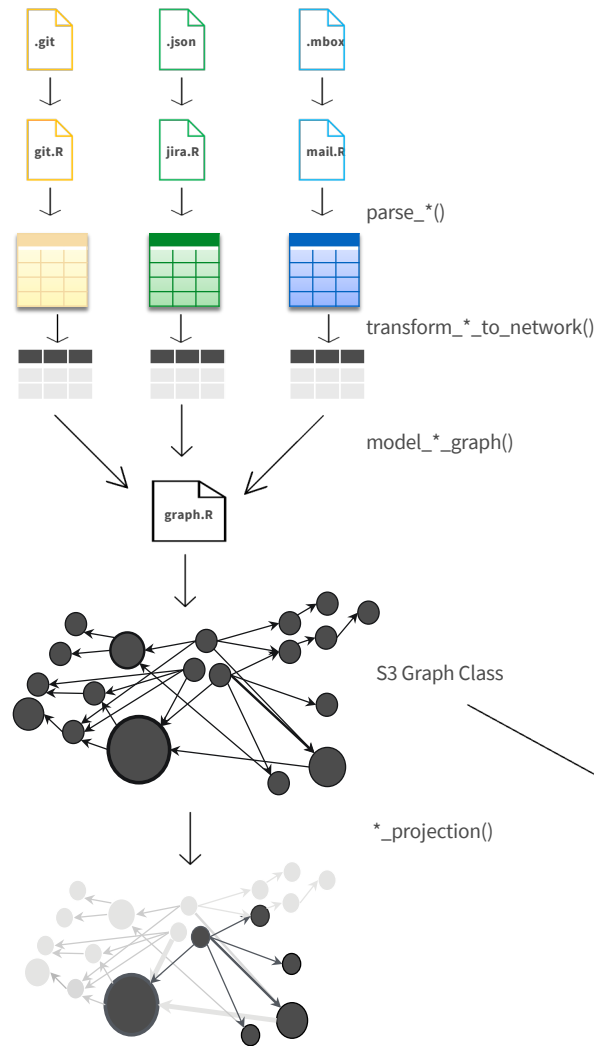
The first part of running any vignette is setting up your project configuration file (examples in `kaiaulu/conf`).

Required Fields

The file `"tools.yml"` must also be configured. See `README.md` for more information on 3rd party software dependencies.

Required Fields:

Network Transform Pipeline



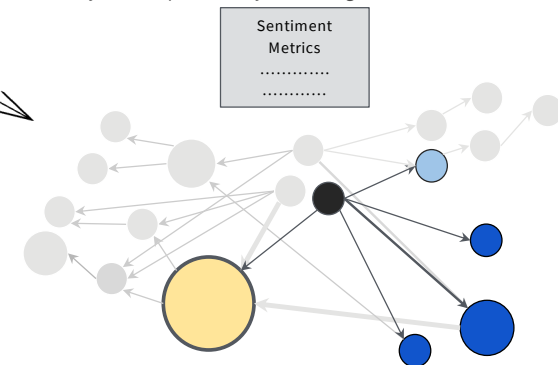
Related Vignettes

See the following notebooks for related analyses:

1. `gitlog_showcase.Rmd`
2. `gitlog_entity_showcase.Rmd`
3. `motif_analysis.Rmd`
4. `community_detection_showcase.Rmd`
5. `blamed_line_types_showcase.Rmd`
6. `line_metrics_showcase.Rmd`
7. `depends_showcase.Rmd`
8. `social_smell_showcase.Rmd`
9. `reply_communication_showcase.Rmd`
10. `issue_social_smell_showcase.Rmd`
11. `gitlog_vulnerabilities_showcase.Rmd`
12. `bug_count.Rmd`
13. `causal_flaws.Rmd`
14. `dv8_showcase.Rmd`

External Weight Integration

The standardized S3 class system makes it straightforward to extend Kaiulu's analysis capabilities by adding new functions that operate on the graph model without touching the transform layer. For example, sentiment metrics can be introduced as a new weight scheme function that receives a typed graph object and is automatically compatible with any network produced by the existing transforms.



Network Transform: : CHEAT SHEET



About

Kaiaulu converts raw data sources into graphs for downstream analysis. This work introduces a standardized S3 class system for Kaiulu's graph model, replacing inconsistent graph construction with two well-defined constructors that encode properties such as directionality, weight, and bipartiteness in the object's class. Consumer functions dispatch on these classes, so new analyses can be implemented as S3 methods that automatically work with any graph the existing transforms produce.

Project Config Setup

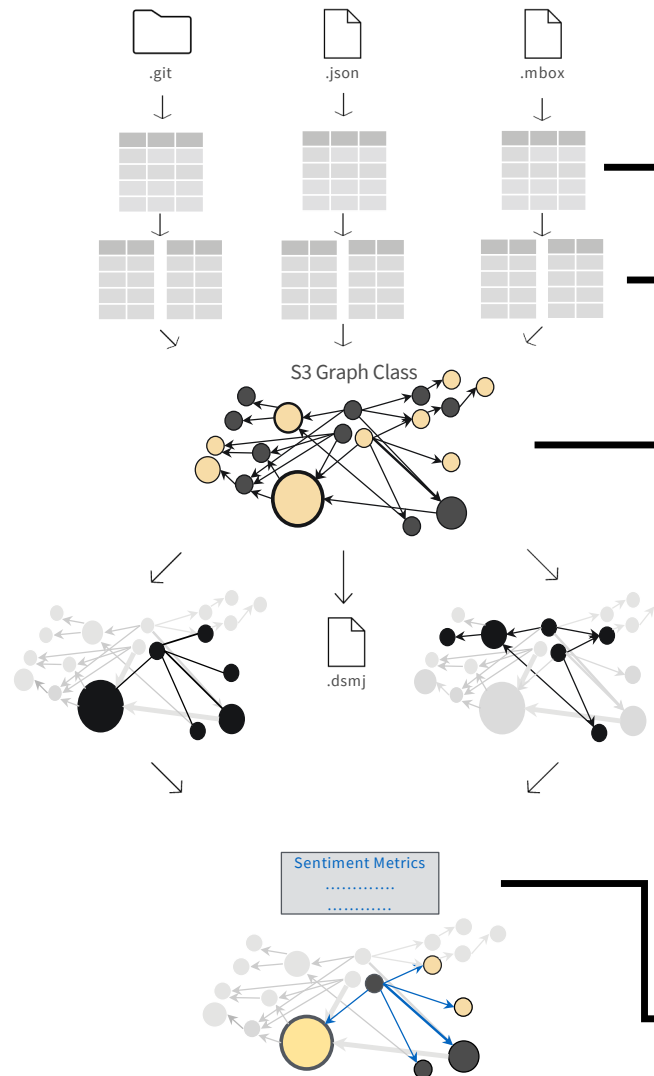
The first part of running any vignette is setting up your project configuration file (examples in `kaiaulu/conf`).

Required Fields

The file "tools.yml" must also be configured. See `README.md` for more information on 3rd party software dependencies.

Required Fields:

Network Transform Pipeline



Network Transform

`parse_*`

Parse functions take raw data sources, such as git logs, mailing lists, issue trackers, and normalize them into structured tables that transform functions can consume. This separation allows the same parsed data to be reused across multiple transforms without re-parsing the source.

`transform_*_to_network()`

Transform functions take parsed data tables and construct graph representations by defining the nodes, edges, and relationships relevant to a specific analysis, such as author-file modifications or temporal developer connections. Each transform returns a standardized S3 graph object ready for downstream analysis.

`model_*_graph()`

1. `model_unimodal_graph()`

Builds S3 class vector for graphs where all nodes are the same type

2. `model_multimodal_graph()`

Builds S3 class vector for graphs with multiple node types such as authors and files.

S3 Functions

Examples of S3 Generic functions:

1. `bipartite_graph_projection()`

Performs a bipartite projection on a multimodal graph

2. `temporal_graph_projection()`

Performs a temporal projection on a multimodal graph

3. `graph_to_dsmj()`

Exports graph to .dsmj file

Examples of S3 Auxiliary Functions:

4. `subset_bipartite_from_multimodal()`

Slices a bipartite graph from a multimodal graph to do projection analyses

External Weight Integration

The S3 class system allows new analyses like sentiment metric weight schemes to be added without modifying any existing transforms.

Network Transform: : CHEAT SHEET



About

Data transformations produce standardized S3 graph objects encoding directionality, weight, and bipartiteness. New analyses can be added as S3 methods that automatically work with any graph the existing transforms produce, without rewriting upstream code.

Project Config Setup

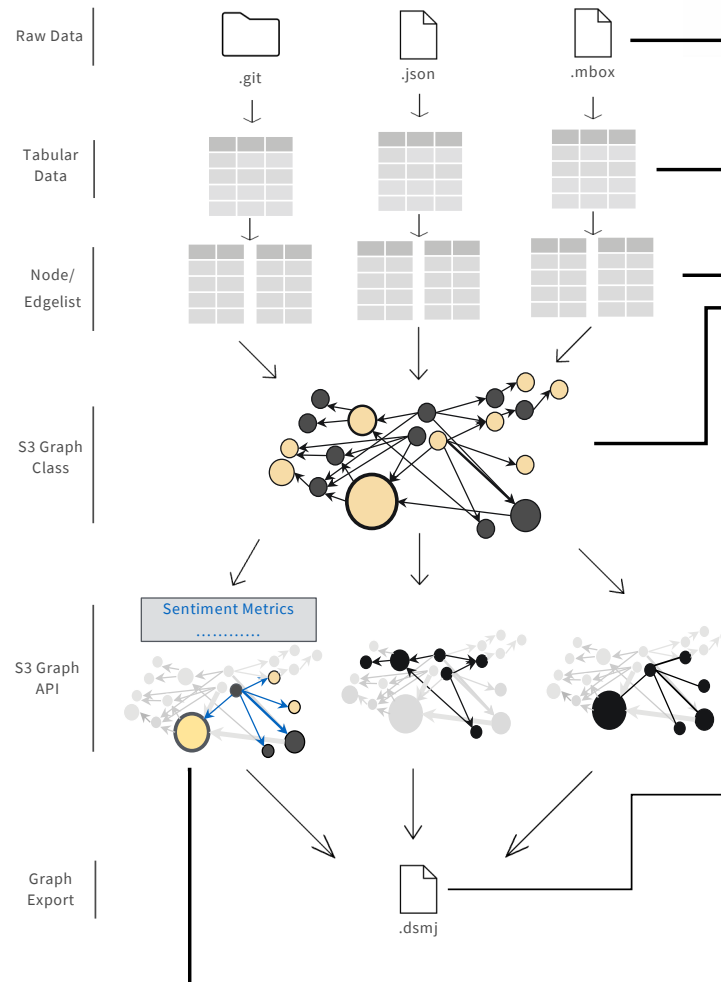
The first part of running any vignette is setting up your project configuration file (examples in `kaiaulu/conf`).

Required Fields

The file `"tools.yml"` must also be configured. See `README.md` for more information on 3rd party software dependencies.

Required Fields:

Network Transform Pipeline



Network Transform

Downloaders

Download raw data: `cheatsheets/refresher-cheatsheet.pdf`

Parsers

Parse functions normalize raw data into structured tables reusable across multiple transforms.

Transformers

Transform functions construct S3 graph objects from parsed tables by defining nodes, edges, and relationships.

Graph Constructors

A graph structure generalizes different types of graphs, regardless of their data source.

1. `model_unimodal_graph()`

Represents a unimodal graph.

2. `model_multimodal_graph()`

Represents a multimodal graph.

S3 Functions

S3 methods execute the correct behavior based on the graph's class, regardless of the data's origin.

Examples of S3 Generic Functions:

1. `bipartite_graph_projection()`

Performs bipartite projection on multimodal graphs.

2. `temporal_graph_projection()`

Performs temporal projection on multimodal graphs.

3. `graph_to_dsmj()`

Exports graph to a `.dsmj` file.

Example of S3 Auxiliary Functions:

4. `subset_bipartite_from_multimodal()`

Slices a bipartite graph from a multimodal graph.

External Weight Integration

The S3 class system allows new analyses like sentiment metric weight schemes to be added without modifying any existing transforms.

1. `transform_reply_to_bipartite_network(weight,...)`

Network Transform: : CHEAT SHEET



About

Data transformations produce standardized S3 graph objects encoding directionality, weight, and bipartiteness. New analyses can be added as S3 methods that automatically work with any graph the existing transforms produce, without rewriting upstream code.

Downloaders

Download raw data: `cheatsheets/refresher-cheatsheet.pdf`

Parsers

Parse functions normalize raw data into structured tables reusable across multiple transforms.

1. `parse_gitlog()`

Parses a git repository into a commit table.

2. `parse_jira_replies()`

Parses Jira issue comments into a reply table.

3. `parse_mbox()`

Parses mbox files into a reply table.

Transformers

Transform functions construct S3 graph objects from parsed tables by defining nodes, edges, and relationships.

1. `transform_gitlog_to_bipartite_network()`

Git Log commit table → bipartite graph.

2. `transform_gitlog_to_multimodal_network()`

Git Log commit table → multimodal graph.

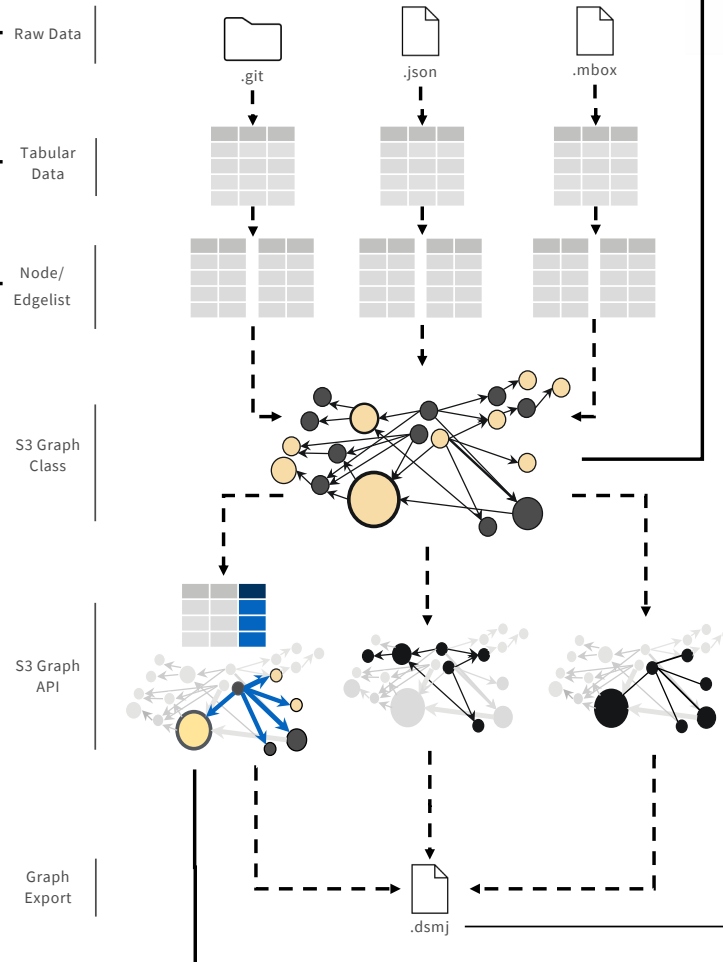
3. `transform_gitlog_to_temporal_network()`

Git Log commit table → temporal projection graph.

4. `transform_reply_to_bipartite_network()`

Reply table → person-issue bipartite graph.

Network Transform Pipeline



Graph Constructors

A graph structure generalizes different types of graphs, regardless of their data source.

1. `model_unimodal_graph()`

Represents a unimodal graph.

2. `model_multimodal_graph()`

Represents a multimodal graph.

S3 Functions

S3 methods execute the correct behavior based on the graph's class, regardless of the data's origin.

Examples of S3 Generic Functions:

1. `bipartite_graph_projection()`

Performs bipartite projection on multimodal graphs.

2. `temporal_graph_projection()`

Performs temporal projection on multimodal graphs.

3. `community_oslom()`

Runs OSLOM community detection on a graph.

4. `graph_to_dsmj()`

Exports graph to a `.dsmj` file.

Example of S3 Auxiliary Functions:

5. `subset_bipartite_from_multimodal()`

Slices a bipartite graph from a multimodal graph.

5. `get_bidirected_edges()`

Returns all mutual edge pairs from a graph.

External Weight Integration

The S3 class system allows new analyses like sentiment metric weight schemes to be added without modifying any existing transforms.

1. `transform_reply_to_bipartite_network(weight, ...)`

2. `transform_gitlog_to_bipartite_network(weight, ...)`

Network Transform: : CHEAT SHEET



About

Raw project data (git logs, mailing lists, issue trackers) is parsed and transformed into S3 (R's object-oriented system) graph objects encoding directionality and weight. Functions like projections, exports, and community detection automatically dispatch the correct behavior regardless of the data origin.

Downloaders

Kaiaulu downloaders allow different raw data to be obtained. For reference, see refreshers [cheatsheet](#): [cheatsheets/refreshers-cheatsheet.pdf](#)

1. `download_jira_issues()`

Downloads JIRA data to a locally as JSON files

Parsers

Parse functions normalize raw data into structured tables reusable across multiple transforms.

1. `parse_gitlog()`

Parses a git repository into a commit table.

2. `parse_jira_replies()`

Parses Jira issue comments into a reply table.

3. `parse_mbox()`

Parses mbox files into a reply table.

Transformers

Transform functions construct S3 graph objects from parsed tables by defining nodes, edges, and relationships.

1. `transform_gitlog_to_bipartite_network()`

Git Log commit table → bipartite graph.

2. `transform_gitlog_to_multimodal_network()`

Git Log commit table → multimodal graph.

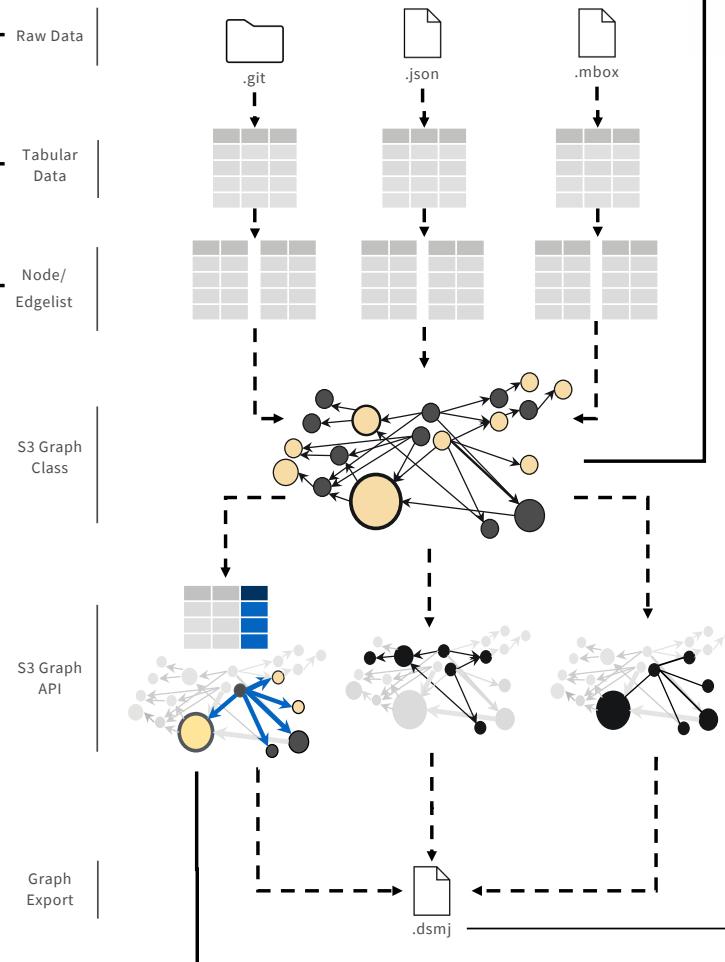
3. `transform_gitlog_to_temporal_network()`

Git Log commit table → temporal projection graph.

4. `transform_reply_to_bipartite_network()`

Reply table → person-issue bipartite graph.

Network Transform Pipeline



Graph Constructors

A graph structure generalizes different types of graphs, regardless of their data source.

1. `model_unimodal_graph()`

Represents a unimodal graph.

2. `model_multimodal_graph()`

Represents a multimodal graph.

S3 Functions

S3 methods execute the correct behavior based on the graph's class, allowing for analyses functions to work regardless of the data's origin.

Examples of S3 Generic Functions:

1. `bipartite_graph_projection()`

Performs bipartite projection on multimodal graphs.

2. `temporal_graph_projection()`

Performs temporal projection on multimodal graphs.

3. `community_oslom()`

Runs OSLOM community detection on a graph.

4. `graph_to_dsmj()`

Exports graph to a `.dsmj` file.

Example of S3 Auxiliary Functions:

5. `subset_bipartite_from_multimodal()`

Slices a bipartite graph from a multimodal graph.

5. `get_bidirected_edges()`

Returns all mutual edge pairs from a graph.

External Weight Integration

Any numeric value — such as sentiment scores or custom weight schemes — can be assigned as edge weights without modifying any existing transforms.

1. `transform_reply_to_bipartite_network(weight, ...)`

2. `transform_gitlog_to_bipartite_network(weight, ...)`

Network Transform: : CHEAT SHEET



About

Kaiāulu unifies a variety of data sources (e.g. git logs, mailing lists) in a consistent object-oriented graph structure using S3 (R's object-oriented system). This abstraction allows future graph functions to be de-coupled from the data source. Graph functions will also apply the correct transformations depending on the graph object being passed as parameter (similarly to a `print()` function).

Downloaders

Kaiāulu downloaders allow different raw data to be obtained. For reference, see refreshers cheatsheet: [cheatsheets/refresher-cheatsheet.pdf](#)

1. `download_jira_issues()`

Downloads JIRA data locally as JSON files.

2. ...

Parsers

Parse functions format raw data into structured tables that are reusable across multiple transforms.

1. `parse_gitlog()`

Parses a git repository into a commit table.

2. `parse_jira_replies()`

Parses Jira issue comments into a reply table.

3. ...

Transformers

Transform functions construct S3 graph objects from parsed tables by defining nodes, edges, and relationships.

1. `transform_gitlog_to_bipartite_network()`

Git Log commit table → bipartite graph.

2. `transform_gitlog_to_multimodal_network()`

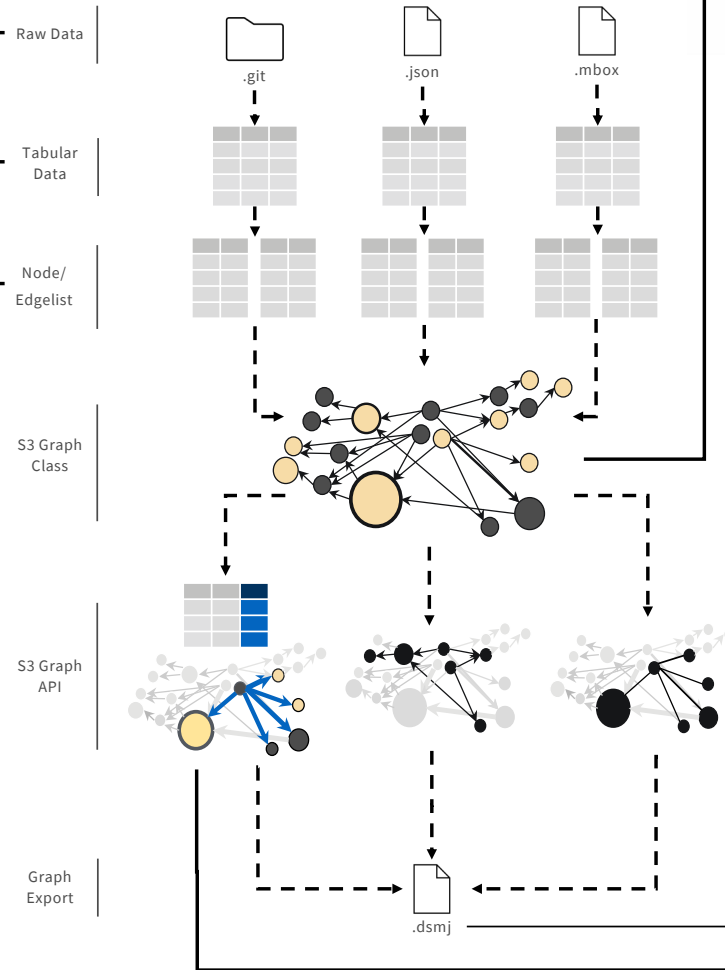
Git Log commit table → multimodal graph.

3. `transform_reply_to_bipartite_network()`

Reply table → person-issue bipartite graph.

4. ...

Network Transform Pipeline



Graph Constructors

Transformers utilize a graph structure which generalizes different types of graphs, regardless of their data source.

1. `model_unimodal_graph()`

Represents a unimodal graph.

2. `model_multimodal_graph()`

Represents a multimodal graph.

S3 Functions

S3 methods execute the correct behavior based on the graph's class, allowing for analyses functions to work regardless of the data's origin.

Examples of S3 Generic Functions:

1. `bipartite_graph_projection()`

Performs bipartite projection on multimodal graphs.

2. `temporal_graph_projection()`

Performs temporal projection on multimodal graphs.

3. `community_oslom()`

Runs OSLOM community detection on a graph.

4. `graph_to_dsmj()`

Exports graph to a `.dsmj` file.

Example of S3 Auxiliary Functions:

5. `subset_bipartite_from_multimodal()`

Slices a bipartite graph from a multimodal graph.

6. `get_bidirected_edges()`

Returns all mutual edge pairs from a graph.

Custom Graph Weights

Any numeric value — such as sentiment scores or custom weight schemes — can be assigned as edge **weights** without modifying any existing transforms.

1. `transform_reply_to_bipartite_network(weight, ...)`

2. `transform_gitlog_to_bipartite_network(weight, ...)`

3. ...

Network Transform: : CHEAT SHEET



About

Kaiāulu's graph module unifies a variety of data sources (e.g. git logs, mailing lists) in a consistent object-oriented structure using S3 (R's object-oriented system). This abstraction allows future graph functions to be de-coupled from the data source. Graph functions will also apply the correct transformations depending on the graph object being passed as parameter (similarly to a `print()` function).

Downloaders

Kaiāulu downloaders allow different raw data to be obtained. For reference, see refreshers cheatsheet: [cheatsheets/refresher-cheatsheet.pdf](#)

1. `download_jira_issues()`

Downloads JIRA data locally as JSON files.

2. ...

Parsers

Parse functions format raw data into structured tables that are reusable across multiple transforms.

1. `parse_gitlog()`

Parses a git repository into a commit table.

2. `parse_jira_replies()`

Parses Jira issue comments into a reply table.

3. ...

Transformers

Transform functions construct S3 graph objects from parsed tables by defining nodes, edges, and relationships.

1. `transform_gitlog_to_bipartite_network()`

Git Log commit table → bipartite graph.

2. `transform_gitlog_to_multimodal_network()`

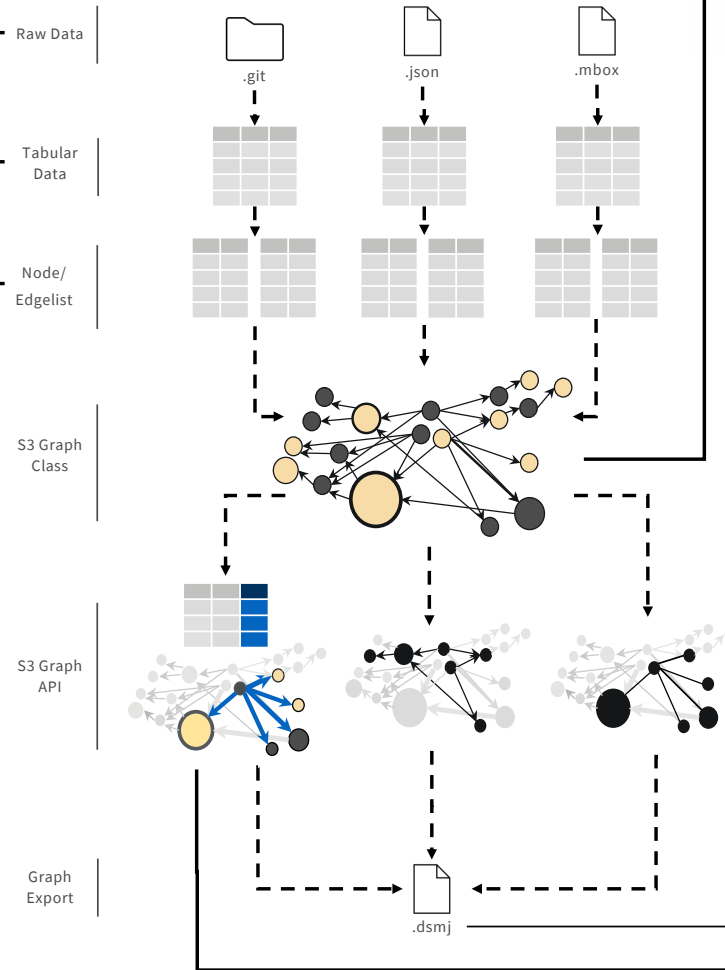
Git Log commit table → multimodal graph.

3. `transform_reply_to_bipartite_network()`

Reply table → person-issue bipartite graph.

4. ...

Network Transform Pipeline



Graph Constructors

Transformers utilize a graph structure which generalizes different types of graphs, regardless of their data source.

1. `model_unimodal_graph()`

Represents a unimodal graph.

2. `model_multimodal_graph()`

Represents a multimodal graph.

S3 Functions

S3 methods execute the correct behavior based on the graph's class, allowing for analyses functions to work regardless of the data's origin.

Examples of S3 Generic Functions:

1. `bipartite_graph_projection()`

Performs bipartite projection on multimodal graphs.

2. `temporal_graph_projection()`

Performs temporal projection on multimodal graphs.

3. `community_oslom()`

Runs OSLOM community detection on a graph.

4. `graph_to_dsmj()`

Exports graph to a .dsmj file.

Example of S3 Auxiliary Functions:

5. `subset_bipartite_from_multimodal()`

Slices a bipartite graph from a multimodal graph.

6. `get_bidirected_edges()`

Returns all mutual edge pairs from a graph.

Custom Graph Weights

Any numeric value — such as sentiment scores or custom weight schemes — can be assigned as edge **weights** without modifying any existing transforms.

1. `transform_reply_to_bipartite_network(weight, ...)`

2. `transform_gitlog_to_bipartite_network(weight, ...)`

3. ...

Network Transform: : CHEAT SHEET



About

Kaiāulu's graph module unifies a variety of data sources (e.g. git logs, mailing lists) in a consistent object-oriented structure using S3 (R's object-oriented system). This abstraction allows future graph functions to be de-coupled from the data source. Graph functions will also apply the correct transformations depending on the graph object being passed as parameter (similarly to a `print()` function).

Downloaders

Kaiāulu downloaders allow different raw data to be obtained. For reference, see refreshers cheatsheet: [cheatsheets/refresher-cheatsheet.pdf](#)

1. `download_jira_issues()`

Downloads JIRA data locally as JSON files.

2. ...

Parsers

Parse functions format raw data into structured tables that are reusable across multiple transforms.

1. `parse_gitlog()`

Parses a git repository into a commit table.

2. `parse_jira_replies()`

Parses Jira issue comments into a reply table.

3. ...

Transformers

Transform functions construct S3 graph objects from parsed tables by defining nodes, edges, and relationships.

1. `transform_gitlog_to_bipartite_network()`

Git Log commit table → bipartite graph.

2. `transform_gitlog_to_multimodal_network()`

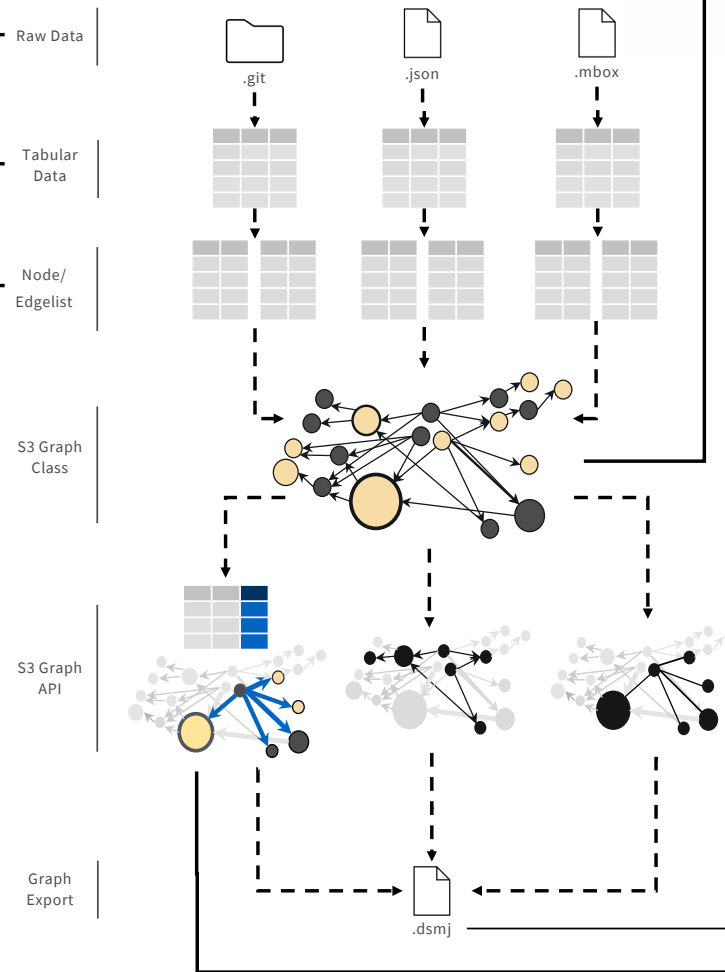
Git Log commit table → multimodal graph.

3. `transform_reply_to_bipartite_network()`

Reply table → person-issue bipartite graph.

4. ...

Network Transform Pipeline



Graph Constructors

Transformers utilize a graph structure which generalizes different types of graphs, regardless of their data source.

1. `model_unimodal_graph()`

Represents a unimodal graph.

2. `model_multimodal_graph()`

Represents a multimodal graph.

S3 Functions

S3 methods execute the correct behavior based on the graph's class, allowing for analyses functions to work regardless of the data's origin.

Examples of S3 Generic Functions:

1. `bipartite_graph_projection()`

Performs bipartite projection on multimodal graphs.

2. `temporal_graph_projection()`

Performs temporal projection on multimodal graphs.

3. `community_oslom()`

Runs OSLOM community detection on a graph.

4. `graph_to_dsmj()`

Exports graph to a `.dsmj` file.

Example of S3 Auxiliary Functions:

5. `subset_bipartite_from_multimodal()`

Slices a bipartite graph from a multimodal graph.

6. `get_bidirected_edges()`

Returns all mutual edge pairs from a graph.

Custom Graph Weights

Any numeric value — such as sentiment scores or custom weight schemes — can be assigned as edge **weights** without modifying any existing transforms.

1. `transform_reply_to_bipartite_network(weight, ...)`

2. `transform_gitlog_to_bipartite_network(weight, ...)`

3. ...